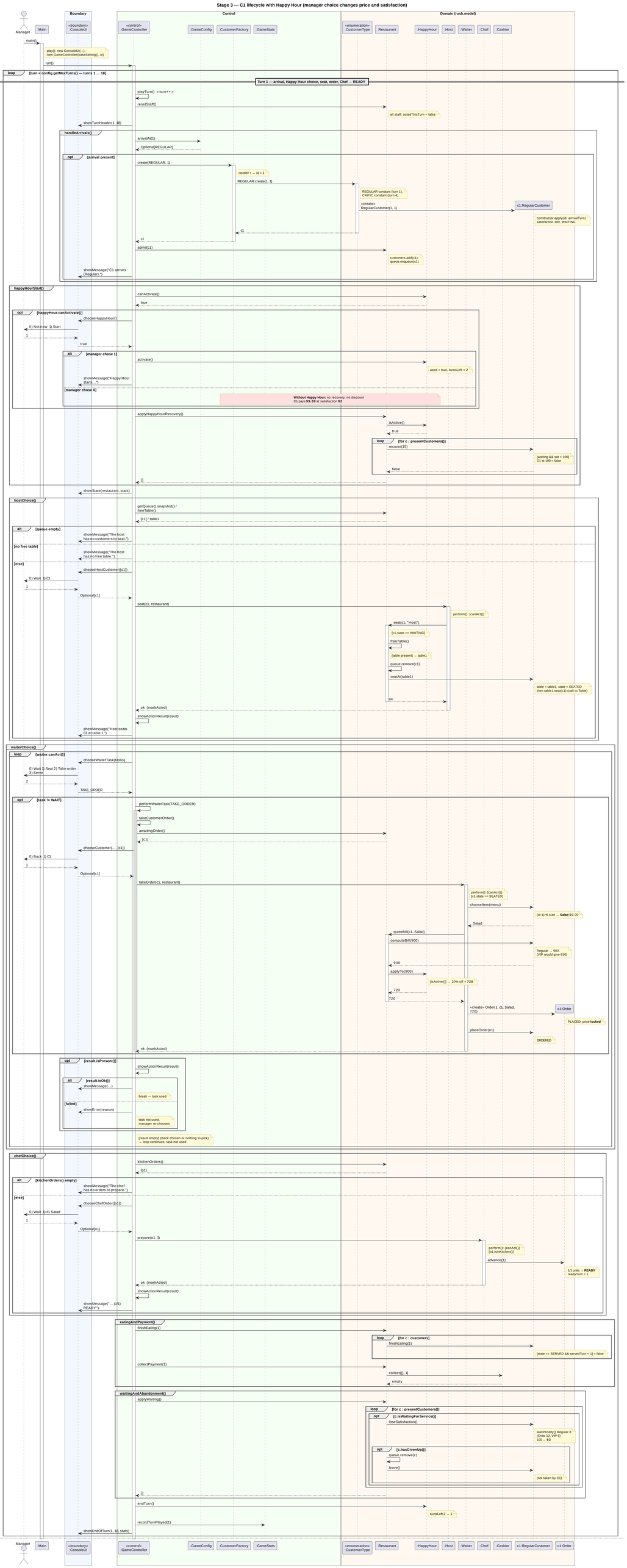
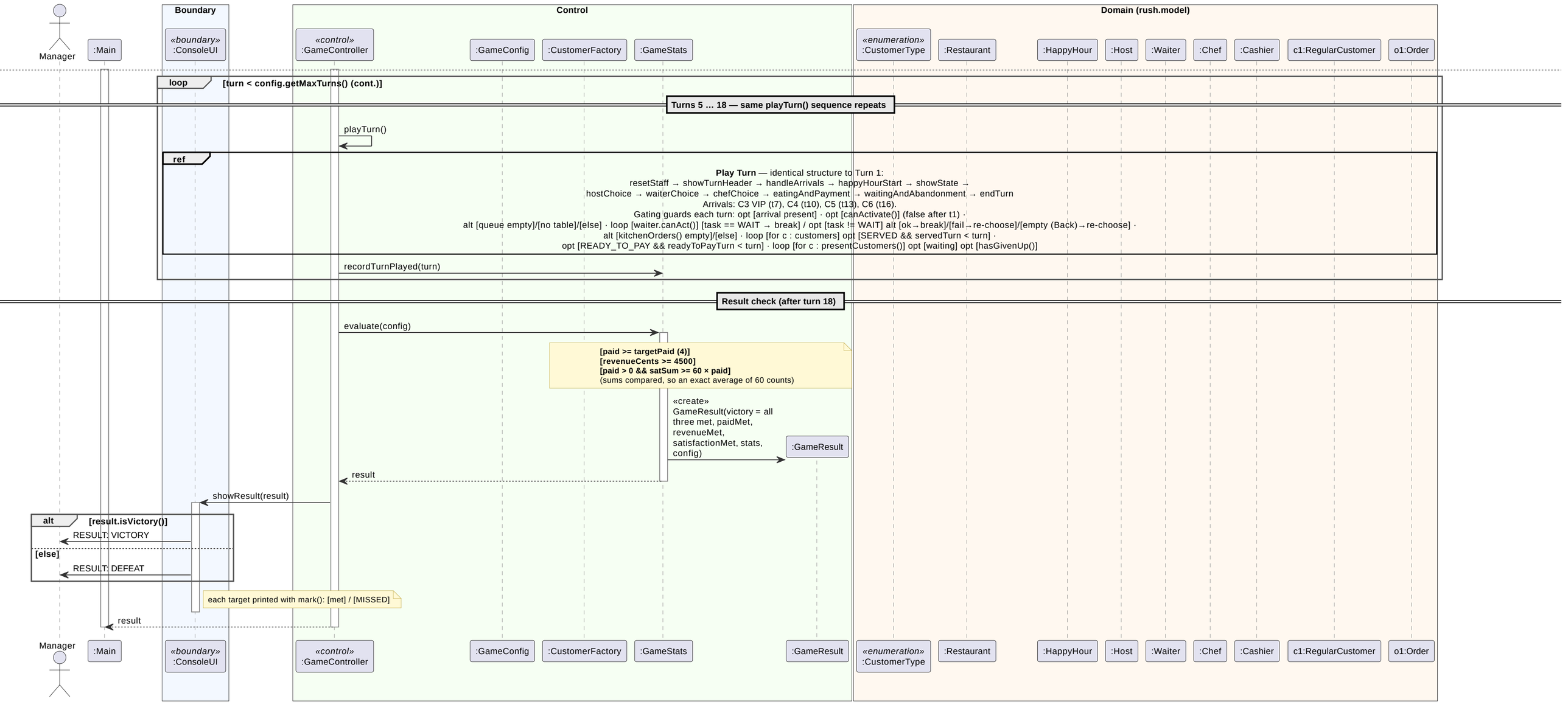
Stage 3 — C1 lifecycle with Happy Hour (manager choice changes price and satisfaction)



Boundary
Control
Domain (rush.model)
Manager
:Main
«boundary» :ConsoleUI
«control» :GameController
:GameConfig
:CustomerFactory
:GameStats
«enumeration» :CustomerType
:Restaurant
:HappyHour
:Host
:Waiter
:Chef
:Cashier
c1:RegularCustomer
o1:Order
loop [turn < config.getMaxTurns() (cont.)]
Turn 2 — Happy Hour recovery, then SERVE
playTurn()
resetStaff()
showTurnHeader(2, 18)
arrivalAt(2) empty → no arrival;
canActivate() = false → no Happy Hour prompt
happyHourStart()
applyHappyHourRecovery()
isActive()
true (turnsLeft 1)
loop [for c : presentCustomers()]
recover(15)
[ORDERED && 92 < 100]
92 → 100
true
[c1]
showMessage("C1 recovers to 100 (Happy Hour).")
showState(restaurant, stats)
showMessage("The host has no customers to seat.")
hostChoice(): [queue empty]
waiterChoice()
loop [waiter.canAct()]
chooseWaiterTask(tasks)
tasks
3
SERVE
opt [task != WAIT]
performWaiterTask(SERVE)
serveOrder()
activeOrders().filter(isReady)
[o1]
chooseOrder(…, [o1])
0) Back 1) #1 Salad [READY]
1
Optional[o1]
serve(o1, 2)
[o1.isReady()] = true
[readyTurn 1 < turn 2] = true
markServed()
SERVED
onServed(2)
SERVED, servedTurn = 2
ok (markActed)
opt [result.isOk()]
recordServed()
opt [result.isPresent()]
showActionResult(result)
showMessage("Waiter serves #1 Salad.")
[result.isOk()] → break — task used
showMessage("The chef has no orders to prepare.")
chefChoice(): [kitchenOrders() empty]
eatingAndPayment()
finishEating(2)
loop [for c : customers]
finishEating(2)
[state == SERVED && servedTurn 2 < 2] = false
collectPayment(2)
empty
waitingAndAbandonment()
applyWaiting()
C1 SERVED → no loss, stays 100
endTurn()
turnsLeft 1 → 0 (over)
recordTurnPlayed(2)
showEndOfTurn(2, 18, stats)
Turn 3 — eating finishes → READY_TO_PAY
playTurn()
resetStaff()
showTurnHeader(3, 18)
no arrival; Happy Hour used → no prompt;
applyHappyHourRecovery() = [] (inactive)
showState(restaurant, stats)
showMessage("The host has no customers to seat.")
hostChoice(): [queue empty]
waiterChoice()
loop [waiter.canAct()]
chooseWaiterTask(tasks)
tasks
0
WAIT
showMessage("The waiter takes no action this turn.")
[task == WAIT] → break
(performWaiterTask not called)
showMessage("The chef has no orders to prepare.")
chefChoice(): [kitchenOrders() empty]
eatingAndPayment()
finishEating(3)
loop [for c : customers]
finishEating(3)
[state == SERVED && servedTurn 2 < 3] = true
READY_TO_PAY, readyToPayTurn = 3
true
collectPayment(3)
collect([c1], 3)
[c1.canPayOn(3)]
[3 < 3] = false, no payment
empty
applyWaiting()
C1 READY_TO_PAY → no loss
endTurn()
recordTurnPlayed(3)
showEndOfTurn(3, 18, stats)

Boundary
Control
Domain (rush.model)
Manager
:Main
«boundary» :ConsoleUI
«control» :GameController
:GameConfig
:CustomerFactory
:GameStats
«enumeration» :CustomerType
:Restaurant
:HappyHour
:Host
:Waiter
:Chef
:Cashier
c1:RegularCustomer
o1:Order
loop [turn < config.getMaxTurns() (cont.)]
Turn 4 — C2 arrives; C1 pays, table freed
playTurn()
resetStaff()
showTurnHeader(4, 18)
handleArrivals()
arrivalAt(4)
Optional[CRITIC]
opt [arrival present]
create(CRITIC, 4)
CRITIC.create(2, 4)
«create» c2:CriticCustomer(2, 4)
c2
admit(c2)
showMessage("C2 arrives (Critic).")
happyHourStart(): Happy Hour used → no prompt;
applyHappyHourRecovery() = [] (inactive)
showState(restaurant, stats)
Host / Waiter / Chef steps for C2 follow the Turn 1 pattern (omitted): Host seats C2 at table 2;
Waiter takes C2's order — CriticCustomer.chooseItem() picks the most prep units (tie → higher price) = Burger+Pasta $29.70, 3 units;
Chef does unit 1/3. Happy Hour is over → C2's order is not discounted (price locked at order time).
eatingAndPayment()
finishEating(4)
loop [for c : customers]
finishEating(4)
[state == SERVED] = false
(C1 is READY_TO_PAY)
collectPayment(4)
readyToPay() → [c1]
collect([c1], 4)
[canAct()] = true
[c1.canPayOn(4)] = [3 < 4] = true
oldest first (min readyToPayTurn, id)
pay()
PAID
markPaid()
PAID
table.release() (call to Table)
table 1 free
markActed()
Optional[c1]
Optional[c1]
opt [customer present]
recordPayment(720, 100)
revenue += 720
satisfactions.add(100)
showMessage("C1 pays $7.20.")
Manager choice changed C1's outcome
Happy Hour ON: $7.20 at satisfaction 100 | OFF: $9.00 at 92
→ trades $1.80 revenue for +8 satisfaction
waitingAndAbandonment()
applyWaiting()
C1 PAID → skipped
C2 loses 12 (Critic)
endTurn()
recordTurnPlayed(4)
showEndOfTurn(4, 18, stats)


Boundary
Control
Domain (rush.model)
Manager
:Main
«boundary»
:ConsoleUI
«control»
:GameController
:GameConfig
:CustomerFactory
:GameStats
«enumeration»
:CustomerType
:Restaurant
:HappyHour
:Host
:Waiter
:Chef
:Cashier
c1:RegularCustomer
o1:Order
loop
[turn < config.getMaxTurns() (cont.)]
Turns 5 … 18 — same playTurn() sequence repeats
playTurn()
ref
Play Turn — identical structure to Turn 1:
resetStaff → showTurnHeader → handleArrivals → happyHourStart → showState →
hostChoice → waiterChoice → chefChoice → eatingAndPayment → waitingAndAbandonment → endTurn
Arrivals: C3 VIP (t7), C4 (t10), C5 (t13), C6 (t16).
Gating guards each turn: opt [arrival present] · opt [canActivate()] (false after t1) ·
alt [queue empty]/[no table]/[else] · loop [waiter.canAct()] [task == WAIT → break] / opt [task != WAIT] alt [ok→break]/[fail→re-choose]/[empty (Back)→re-choose] ·
alt [kitchenOrders() empty]/[else] · loop [for c : customers] opt [SERVED && servedTurn < turn] ·
opt [READY_TO_PAY && readyToPayTurn < turn] · loop [for c : presentCustomers()] opt [waiting] opt [hasGivenUp()]
recordTurnPlayed(turn)
Result check (after turn 18)
evaluate(config)
[paid >= targetPaid (4)]
[revenueCents >= 4500]
[paid > 0 && satSum >= 60 × paid]
(sums compared, so an exact average of 60 counts)
«create»
GameResult(victory = all
three met, paidMet,
revenueMet,
satisfactionMet, stats,
config)
:GameResult
result
showResult(result)
alt
[result.isVictory()]
RESULT: VICTORY
[else]
RESULT: DEFEAT
each target printed with mark(): [met] / [MISSED]
result